2020 年同等学力申硕计算机综合试题-软件工程

一、单项选择题（本大题共 5 小题，每题 1 分，共 5 分）

1．一个模块内部的各部分没什么联系，另一个模块通过传递基本参数向它传递一个控制，该模块属于什么（ B ）内聚

A. 数据内聚
B. 逻辑内聚
C. 通信内聚
D. 顺序内聚

解析：考察软件设计-结构化设计

内聚是衡量模块内部各元素（语句、功能等）之间关联程度的指标，内聚程度从低到高可分为多种类型：

- **数据内聚：**模块内各部分通过数据传递关联，共同处理同一批数据，内聚程度较高。
- **逻辑内聚：**模块内各部分逻辑上相关（如多个相似功能的集合），但功能独立性差，内部联系松散。题目中 “模块内部的各部分没什么联系” 且接收控制参数（用于选择执行不同逻辑），符合逻辑内聚的特征。
- **通信内聚：**模块内各部分使用相同的输入数据或产生相同的输出数据，关联基于数据通信，内聚程度中等。
- **顺序内聚：**模块内各部分按顺序执行，前一部分的输出是后一部分的输入，关联紧密，内聚程度高。
- 题干中模块内部元素联系弱，且通过控制参数决定执行逻辑，属于典型的逻辑内聚。

答案：B. 逻辑内聚。

2．DFD 顶层流图用于指出系统的（ C ）

A. 数据主体
B. 数据存储
C. 输入与输出
D. 处理过程

解析：考察软件设计-结构化分析方法

数据流图（DFD）的顶层流图，也称为上下文图，它将整个系统看作一个单一的处理过程，主要作用是展示系统与外部实体之间的交互，明确系统的输入来源和输出去向，也就是指出系统的输入与输出。

答案：C. 输入与输出。

3．黑盒测试主要考虑系统的（ B ）

A. 程序流程
B. 功能
C. 判定分支结构
D. 异常处理

淘宝店：PandaCode辉文软件编程库 原创

解析：考察软件测试

黑盒测试是把测试对象看作一个黑盒子，测试人员完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求规格说明书，检查程序的功能是否符合它的功能说明。它主要关注的是系统的功能实现情况，而不是程序内部的流程、判定分支结构或者特定的异常处理细节（异常处理可作为功能的一部分进行测试，但不是黑盒测试的核心考量维度）。

答案：B. 功能。

4. RUP 中的‘谁做’，‘怎么做’，‘做什么’里的‘谁做’有哪个概念/术语（ B ）完成

A. 制品
B. 角色
C. 工作流
D. 数据

解析：考察软件过程

- **A. 制品：**在 RUP（统一软件开发过程）中，制品是软件开发过程中产生的各种工作成果，比如文档、模型、代码等，它主要体现“做什么”产生的结果，而不是“谁做”，所以 A 选项错误。
- **B. 角色：**角色定义了在软件开发过程中不同人员或团队的职责和分工，明确了某项任务由谁来执行，对应“谁做”这一概念，所以 B 选项正确。
- **C. 工作流：**工作流描述了软件开发过程中活动的顺序和流程，侧重于“怎么做”，即规定了完成任务的步骤和方式，并非“谁做”，所以 C 选项错误。
- **D. 数据：**数据是软件开发过程中涉及的信息载体，如需求数据、设计数据等，与“谁做”没有直接关联，所以 D 选项错误。

答案：B. 角色。

5. 面向对象设计模式，说法正确的是（ B ）

A. 使程序员能够更快的完成
B. 可以复用好的设计
C. 使程序以最快的速度
D. 仅适用于特定编程语言

解析：考察软件设计-面向对象方法

- **A 选项：**设计模式的核心是复用设计思想，而非直接加速编码速度，编码效率受多种因素影响，故 A 错误。
- **B 选项：**设计模式是对反复出现的问题的成熟解决方案，本质是复用经过验证的设计思路，减少重复设计，故 B 正确。
- **C 选项：**设计模式侧重代码的可维护性、扩展性，与程序运行速度无直接关联，甚至部分复杂模式可能略微影响性能，故 C 错误。
- **D 选项：**设计模式是通用设计层面的思想，不依赖特定编程语言（如单例模式可在 Java、Python 等多种语言中实现），故 D 错误。

答案：B. 可以复用好的设计。

淘宝店：GitHubCode 代码文档源码库，原创

二、判断题（本大题共 5 小题，每题 1 分，共 5 分。假如对的，用“√”表达，否则，用“×”表达）

1. CMMI 是过程改进模型，它支持阶段性过程改进不支持连续性过程改进（×）

解析：考察软件工程管理

CMMI（能力成熟度模型集成）是一种用于过程改进的模型，它同时支持阶段性过程改进，也支持连续性过程改进：

- **阶段性改进：**通过成熟度等级（从 1 级到 5 级）描述组织过程的进化路径，强调按阶段组织整体过程能力的提升。
- **连续性改进：**通过过程域的能力等级（从 0 级到 3 级）评估单个过程域的改进，允许组织根据自身需求选择特定过程域进行优化。

因此，题干中 “不支持连续性过程改进” 的说法错误。

2. 某项目组织打算开发一个大型系统，且具备了相关领域知识以及开发经验，此时采用瀑布模型是最好的选择（×）

解析：考察软件过程

瀑布模型的核心是按 “需求分析→设计→编码→测试→维护” 的线性阶段推进，要求需求在初期完全明确且稳定。虽然题干中提到 “具备相关领域知识和开发经验”，但大型系统通常存在需求复杂、易变的特点：

- 即使有经验，大型系统的需求也可能在开发过程中因业务变化、用户反馈等发生调整，瀑布模型难以适应这种变化，修改成本高。
- 对于大型系统，更适合采用迭代模型（如 RUP）、敏捷模型等，通过分阶段交付和持续反馈应对需求变化，降低风险。

因此，“采用瀑布模型是最好的选择” 说法错误。

3. 采用 UML 进行软件设计时，可用依赖关系表示两类事物的“整体/部分”关系（×）

解析：考察软件设计-面向对象方法

在 UML（统一建模语言）中，不同关系有明确的语义和用途，不可混淆：

- **依赖关系：**表示一个事物（客户端）依赖另一个事物（供应端）的功能或服务，客户端的实现需要供应端支持（如 “类 A 调用类 B 的方法”），是一种弱关联，不体现 “整体 / 部分” 的包含或组成关系。
- **“整体 / 部分” 关系：**需通过 UML 的**聚合关系**（整体与部分可独立存在，如 “班级与学生”）或**组合关系**（整体与部分不可分割，如 “汽车与发动机”）表示，二者均属于关联关系的特殊形式，而非依赖关系。

因此，题干中 “用依赖关系表示‘整体 / 部分’关系” 的说法错误。

4. 质量控制不是软件配置管理活动（√）

解析：考察软件工程管理

软件配置管理（SCM）和软件质量控制（QC）是软件工程中两个独立且目标不同的活动：

- **软件配置管理活动：**核心是对软件生命周期中的配置项（如代码、文档、数

淘宝店：PandaCode 程序源码库，原创

据）进行标识、控制、审计和报告，确保配置项的完整性、一致性和可追溯性，具体包括配置标识、版本控制、变更控制、配置审计等。

- **软件质量控制活动：**核心是通过测试、评审等手段验证软件是否符合质量要求（如功能正确性、性能、可靠性），识别并修正质量缺陷，具体包括测试（黑盒 / 白盒测试）、代码评审、质量度量分析等。

二者目标和范畴不同，质量控制不属于软件配置管理的活动范畴，因此题干说法正确。

5. 风险预测从两个方面，即风险发生的可能性跟风险消除方法（×）

解析：考察软件工程管理

风险预测（也称为风险评估）的核心是识别和分析潜在风险对项目的影响，其关键两个维度是风险发生的可能性（概率） 和风险发生后的影响程度（后果严重性） ，而非 “风险消除方法”：

- **“风险发生的可能性”：**评估风险事件发生的概率高低（如 “需求变更的概率为 30%”）；
- **“风险影响程度”：**评估风险若发生，对项目范围、进度、成本、质量等目标的破坏程度（如 “严重影响开发进度，导致延期 2 周”）；
- **“风险消除方法”：**属于风险应对阶段的内容，是在风险预测后，针对已识别的风险制定的处理策略（如规避、转移、缓解风险），与风险预测的核心维度无关。

因此，题干中 “风险预测从风险发生的可能性跟风险消除方法两个方面” 的说法错误。

三、简答题（本大题共 3 小题，每题 4 分，共 12 分）

1. 什么是开源软件？试举一简单说明。

解析：考察软件工程概述

(1)开源软件：指源代码可以被公众自由获取、查看、修改、分发的软件，其核心特征是遵循开源许可协议（如 GPL、MIT 协议等），允许用户在协议规定范围内对软件进行二次开发、传播，且通常不收取核心功能的使用费用，同时强调社区协作开发和透明化的代码管理。

(2)举例：Linux 操作系统，其源代码完全开放，全球开发者可参与代码修改与优化，用户可自由下载、安装并根据需求定制（如用于服务器、嵌入式设备等场景），且遵循 GPL 开源协议，确保修改后的衍生版本也需保持开源特性。

2. RUP 的每一个迭代周期中设计 6 个核心工程工作流，3 个核心支持工作流。试简述每个核心工程工作流的名称和主要目的。

解析：考察软件过程

RUP（统一软件开发过程）的每个迭代周期中，6 个核心工程工作流及其主要目的如下：

- **1.业务建模：**通过建立业务用例模型，理解组织的业务流程、目标和组织结构，明确系统需要解决的业务问题，为后续开发提供业务上下文。

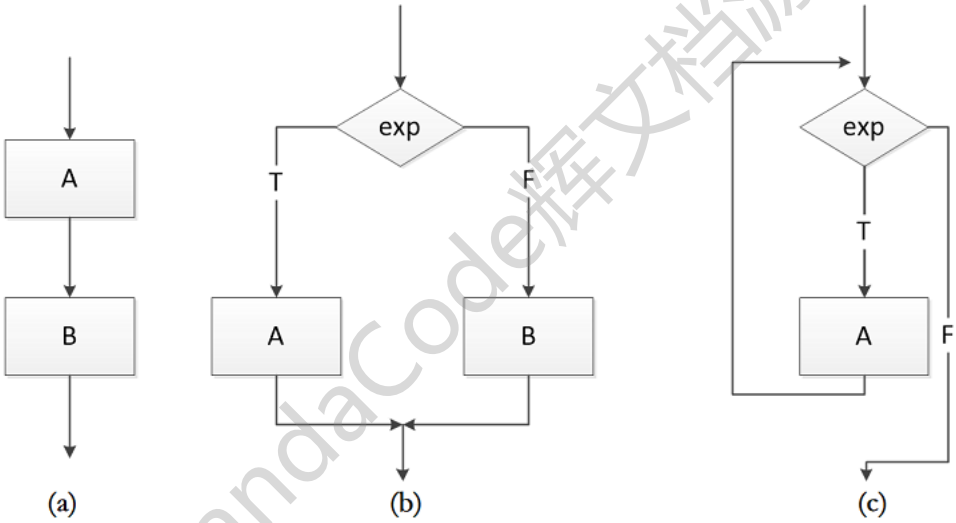
淘宝店：PandaCode辉文程序码库 原创

- **2. 需求：**收集、分析和文档化用户需求，建立用例模型和补充需求规格，明确系统必须实现的功能和非功能需求，作为设计和测试的依据。
- **3. 分析与设计：**将需求转化为系统的设计模型，包括架构设计、类设计、接口设计等，优化系统结构以满足需求，同时平衡性能、可扩展性等质量属性。
- **4. 实现：**根据设计模型编写代码，将设计转化为可执行的软件组件，进行单元测试和集成测试，确保代码符合设计规范。
- **5. 测试：**验证软件是否满足需求，包括单元测试、集成测试、系统测试和验收测试，识别并修复缺陷，确保软件质量。
- **6. 部署：**将经过测试的软件系统交付给用户，包括安装、配置、用户培训和数据迁移等，确保系统在实际环境中正常运行。

3. 结构化程序设计方法要求再复杂的程序结构也应使用基本控制结构组合嵌套来实现。试问这些基本控制结构有哪些？并试用类 C 语言或程序流程图简单描述。

解析：考察软件设计-结构化设计

答：结构化程序设计的基础控制结构有顺序结构、选择结构、循环结构，每个只有一个入口和出口。如下程序流程图描述。



(a)顺序结构，先执行A再执行B；(b)IF-THEN-ELSE 型选择(分支)结构；
(c)DO-WHILE 型循环结构

四、建模题（本大题共 3 小题，共 8 分）

在学校的教学管理系统中，学生查询成绩就是系统中的一次交互行为。问题陈述如下：开始时终端上显示首页，用户选择“查询”请求后，显示“请输入学号”。在用户输入学号后，系统查询学生文件，和对学生学号：若输入的学号不正确，则显示“输入学号不正确”，此次查询取消，回到首页；若输入正确，出现“请输入课程名”。

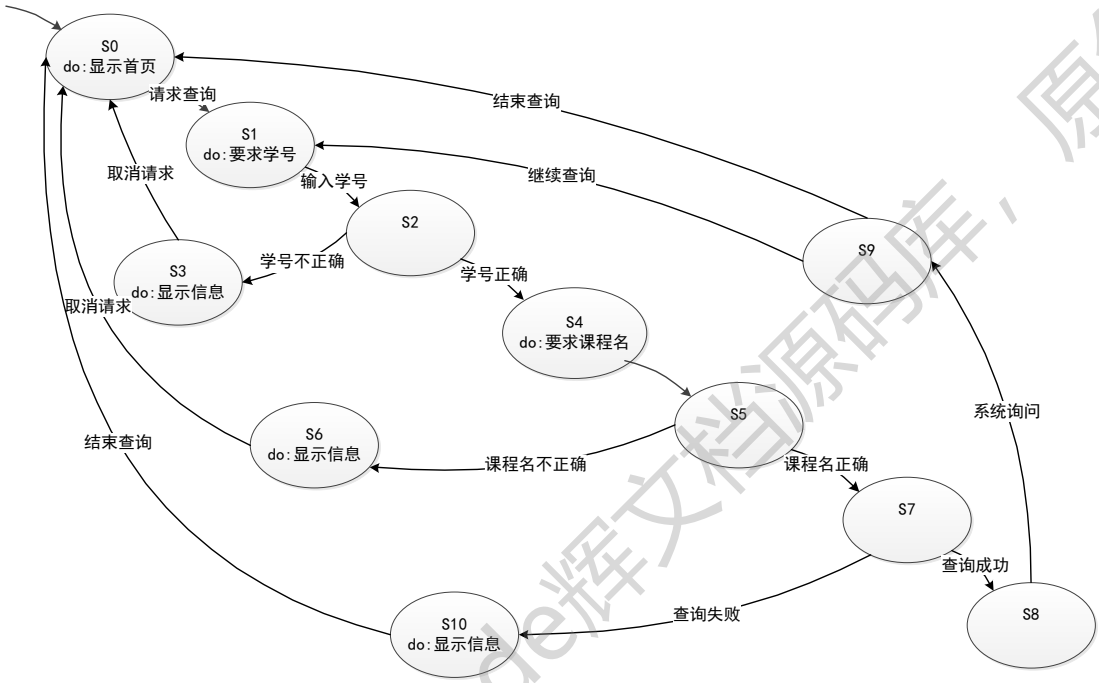
一旦输入课程名，则开始查找课程文件：若输入的课程名不正确，则显示“输入

淘宝店：PandaCode辉文档源码库，原创

的课程名不正确”，此次查询取消，回到首页；若输入正确，则根据“学号”和“课程名”查询选课文件。

若在选课文件中查询成功，显示查询到的成绩，当用户选择“继续查询”后回到“请输入学号”，当用户选择“结束查询”后回到首页。若查询失败，则显示“查询失败”后回到首页。

1. （5 分）下图是该系统的状态图，其中有 5 个状态的名字缺失，请补充完整。
2. （3 分）计算途中的 McCabe 环路复杂度，并依据基本路径覆盖原则确定可能的测试路径。



解析：考察软件设计-面向对象方法与 UML 状态图

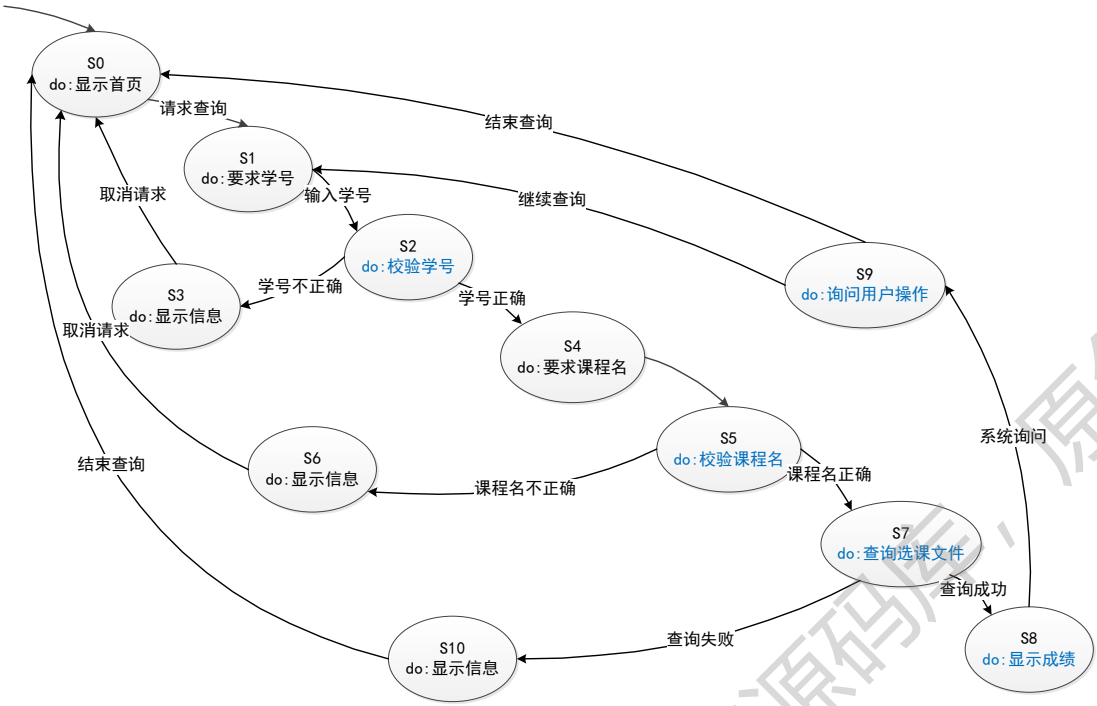
问题 1：补充状态图中缺失的状态名：

根据问题描述和状态图的逻辑，缺失状态的补充如下：

- S2：**校验学号**（系统对输入的学号进行校验的状态）。
- S5：**校验课程名**（系统对输入的课程名进行校验的状态）。
- S7：**查询选课文件**（系统根据学号和课程名查询选课文件的状态）。
- S8：**显示成绩**（查询成功后显示成绩的状态）。
- S9：**询问用户操作**（显示成绩后，询问用户是继续查询还是结束查询的状态）。

补齐的系统状态图如下：

淘宝店 Code辉文档源码库，原创



问题 2：计算 McCabe 环路复杂度并确定测试路径：

（1）计算 McCabe 环路复杂度

McCabe 环路复杂度有多种计算方法，这里采用“**判定节点数 + 1**”的方法（判定节点是指有分支判断的节点，如判断学号是否正确、课程名是否正确、查询是否成功等）。

从状态图中可以看出，判定节点有：学号是否正确、课程名是否正确、查询是否成功、用户选择（继续查询/结束查询），共 4 个判定节点。

因此，环路复杂度 = 判定节点数 + 1 =4 + 1 = 5 。

（2）确定可能的测试路径（基于基本路径覆盖原则）

基本路径覆盖要求设计足够的测试用例，使得程序中每一条独立的路径都至少被执行一次。结合状态图的逻辑，可能的测试路径如下：

- **路径 1：** S0 → S1 → S2 → S3 → S0 （输入学号不正确，查询取消返回首页）。
- **路径 2：** S0 → S1 → S2 → S4 → S5 → S6 → S0 （输入课程名不正确，查询取消返回首页）。
- **路径 3：** S0 → S1 → S2 → S4 → S5 → S7 → S10 → S0 （查询选课文件失败，返回首页）。
- **路径 4：** S0 → S1 → S2 → S4 → S5 → S7 → S8 → S9 → S0 （查询成功，用户选择结束查询返回首页）。
- **路径 5：** S0 → S1 → S2 → S4 → S5 → S7 → S8 → S9 → S1 （查询成功，用户选择继续查询，回到要求学号的状态）。

淘宝店：Paascode毕设文档源码库，原创